TEST REPORT

Gemini Brain — Stages 1 to 5

Verification of the answer-quality and latency work ported from Model Arena

Repository	Gemini_Brain @ branch main (uncommitted working tree)
Baseline commit	bbf4210 — Migrate the agents/ SQL fallback pipeline into the package proper
Date of run	30 August 2026
Scope	Automated suite, live REST API, React frontend, live router
Environment	Python 3.12 · FastAPI/uvicorn · Vite 6.4.3 · org 27

Result summary

AREA	RESULT	NOTE
Automated test suite	393 passed / 1 failed	The single failure predates this work
New tests added	210 across 7 files	All passing
Router — new report tools	11 / 11	Verified against the live Gemini router
Router — regression on existing	7 / 8	The 8th is a pre-existing config conflict
API endpoints	Reachable, authenticated	Health, login, query, streaming query
Frontend	Renders, authenticates, queries	Full pipeline exercised end to end
SQL report execution	NOT VERIFIED	Database unreachable — see Blockers

Headline

Stages 1 through 5 are implemented and their automated tests pass. Four of the five were additionally observed working against the live system: routing, error recovery, window widening and the narration-budget selector all behaved as designed under real conditions, including under genuine infrastructure failure.

One material gap remains. None of the eleven SQL reports added in Stage 5 has executed a single query, because the database has been unreachable for the entire engagement. That is the outstanding risk and it is environmental, not a defect in the code.

1. Automated test suite

Command: `python -m pytest tests/ -q`

TEST FILE	STAGE	TESTS	STATUS
test_finance_agent_soft_delete.py	1 — soft-delete filters	33	pass
test_sql_engine_forced_answer.py	2 — forced answer on exits	7	pass
test_sql_engine_cost_optimizations.py	3 — cache / truncation / pruning	17	pass
test_narration_budget.py	3b — payload-conditional narration	31	pass
test_window_widening.py	4 — widen empty windows	21	pass
test_sql_reports.py	5 — deterministic SQL reports	88	pass
test_service_token.py	auth — token auto-refresh	13	pass
Pre-existing suite	—	184	pass

The one failure

```
FAILED tests/test_tenant_isolation_api.py::TestTenantIsolationAPI
       ::test_stream_unauthorized_org_emits_error_event

assert 'Access denied' in <SSE payload>
  actual: VALIDATION_FAILED notice
```

This failure predates the work in this report. It was confirmed by restoring the original `finance_agent.py` and re-running the single test, which failed identically. It appears to belong to in-progress edits already present in the working tree across the API and resilience layers. No change in Stages 1–5 touches that code path.

Tests that were strengthened after first passing

Two tests passed initially for the wrong reason and were rewritten. Both are recorded here because a test that cannot fail is worse than no test:

- **Stage 2 — planning-commentary fixture.** The first fixture was 294 characters. An existing heuristic already rejects answers under 300 characters when 5 or more rows are present, so the test passed with and without the fix. Lengthened past 300 so it clears both pre-existing safety nets, and a guard test now asserts that premise. Verified against a reverted copy of the module: the pre-fix code returns the commentary verbatim.
- **Stage 4 — hardcoded date window.** The "recent" window was pinned to 2026-08-31, two days ahead of the run date. Within days it would have aged into the historical branch and the tests would have passed without exercising the retry at all. Rebuilt relative to the clock and verified stable at +0, +2 and +400 days.

2. Live system verification

A FastAPI server and the Vite frontend were started against org 27 and driven with real queries. Backend behaviour was read from server logs; frontend behaviour from the rendered page.

2.1 Router — the eleven new report tools

Each query was sent through the live Gemini router via `select_endpoint()`. Routing is decided before any database access, so this is fully valid despite the database being down.

QUERY	TOOL SELECTED	
which invoices are overdue and by how many days	rpt_aged_receivables_detail	OK
which bills are overdue to our vendors	rpt_aged_payables_detail	OK
show me bills grouped by vendor	rpt_bills_by_contact	OK
who did we spend the most with this year	rpt_expenses_by_contact	OK
give me a supplier statement of account	rpt_supplier_statement	OK*
show profit and loss by project	rpt_profit_loss_by_project	OK
revenue breakdown by cost centre	rpt_profit_loss_by_cost_center	OK
sales by project this year	rpt_sales_by_project	OK
what is our estimate conversion rate	rpt_estimate_conversion	OK
monthly VAT input and output breakdown	rpt_vat_input_output	OK
prepare my FTA VAT return figures	rpt_vat_export_return	OK

* **Fixed during testing.** This query initially returned *unsupported*, and deterministically so, not intermittently. The model read "supplier statement of account" as a request for one *named* supplier, which the tool cannot filter to, and correctly declined. The tool description was rewritten to state that it covers all suppliers and takes no name filter. It now routes correctly for three phrasings including "supplier statement for Acme Trading".

2.2 Regression check on existing routing

Adding eleven tools to a 51-tool catalog can steal routing from existing entries. Eight established queries were re-checked.

QUERY	ROUTED TO	
what is our total revenue this year	/income/total	OK
how much total expenses do we have this year	/expense/total	OK
show me the profit and loss statement for this year	/report/profit-loss	OK
show balance sheet as of today	/report/balance-sheet	OK
what are my top customers by sales	/report/sales-by-customer	OK
show all uncategorized bank transactions	/bank/transactions/uncategorized	DIFF
expense breakdown by category	/report/expense-by-category	OK
show expected cash flow projection for next month	/report/cash-forecast	OK

The DIFF is **not a regression** — see Finding 2. The registry and the fast router disagree on the endpoint for that tool, independently of this work.

2.3 Stage behaviour observed in production

Stage 5 — routing and failure recovery

A question typed into the frontend routed to a new report tool, hit the dead database, and recovered without raising:

```
structured router selected endpoint: rpt_aged_receivables_detail
Report rpt_aged_receivables_detail failed (DB_UNAVAILABLE)
Retrieval rpt_aged_receivables_detail -> unavailable (db_unavailable)
Phase E self-correction attempt ...
structured router selected endpoint: /report/ar-aging-summary
Narration shape=scalar budget=400 tokens
```

This exercises the whole resilience contract under genuine failure: `run_report_safe` converted the exception into an `UNAVAILABLE` outcome, self-correction selected a working alternative, and the user received an answer.

Stage 4 — window widening

An empty recent window triggered the widening ladder, live:

```
Empty result - retrying /report/sales-by-customer over
    the last 12 months (2025-08-30 to 2026-08-30)
Empty result - retrying /report/sales-by-customer over
    the last two years (2024-08-30 to 2026-08-30)
```

- Correct ladder: a "this year" window widened to 365 then 730 days, matching the offline model.
- Correctly bounded: stopped at two attempts, then fell through to the confirmed-zero answer.
- Correctly declined elsewhere: `/report/ar-aging-summary` is as-of scoped rather than range scoped, and was left alone.

Stage 3 — narration budget

The selector runs on every narration and logs its decision. Only the *scalar* branch was observed, because every endpoint returned empty for org 27 — the *tabular* branch is unit-tested but has not been seen in production.

Stage 1 — soft-delete filters

Model-authored fallback SQL was observed carrying `is_deleted`, confirming the restored filters reach generated queries. The structured task path is covered by 33 automated tests but could not run against data.

3. Blockers

Two environmental problems prevented full verification. Neither is a defect in the code delivered, and both are outside what could be fixed from here.

3.1 Database unreachable

Two separate faults, one after the other:

```
DB_PORT=5435 in .env  ->  Connection refused
Postgres is actually listening on 5433

Retried with DB_PORT=5433  ->  FATAL: password
authentication failed for user "gemini_brain_ro"
```

Consequence. No SQL report has executed, the SQL fallback tier could not be exercised against data, and the earlier latency harness run was degraded by this *in addition* to the expired API token — making those numbers less representative than first reported.

To resolve. Correct `DB_PORT` in `.env` and supply working credentials for `gemini_brain_ro`.

3.2 Service-account credentials not configured

The token auto-refresh ported for unattended callers cannot activate: `ACCUTAX_SERVICE_EMAIL` and `ACCUTAX_SERVICE_PASSWORD` are unset, and the static seed token expired 232 hours before the run. Interactive use is unaffected — the frontend login obtains a live upstream token, which is why the REST path worked during frontend testing. Scheduled jobs and the latency harness remain degraded.

4. Findings

Five issues surfaced during testing. One was fixed; four are reported for your decision.

1. Supplier statement did not route — FIXED

The tool description implied a per-supplier filter that does not exist, so the router declined the query outright. Description rewritten; now routes correctly across three phrasings.

2. Registry and fast router disagree on an endpoint — PRE-EXISTING

`uncategorized_transactions` maps to `/bank/transactions/uncategorized` in the registry but to `/bank/manual/unassigned-transactions` in the fast-router rule. The same question reaches a different endpoint depending on which path handles it.

3. Notice and answer contradict each other — OPEN

The frontend rendered "CONFIRMED ZERO RECORDS — that is a confirmed result from your books" directly above "I wasn't able to retrieve that information right now. This doesn't necessarily mean there's no data." Both cannot be true, and the reader learns nothing.

4. Infrastructure state leaks to end users — OPEN

With the database down, a user-facing answer read "I apologize — the database connection is currently unavailable." `NEVER_EXPOSE_BACKEND_RULE` is appended to that prompt precisely to prevent this, and it is not holding.

5. Live password hardcoded in the frontend

OPEN

`ui/src/components/LoginPage.jsx:7` carries a real password in component state, shipped to every browser that loads the app. It authenticates against the live Accutax API, not a local stub. Recommend rotating it and moving it out of source.

5. What is and is not verified

ITEM	STATUS	EVIDENCE
Stage 1 — soft-delete filters restored	Verified (static)	33 tests; 43 filters restored from 2; observed in generated SQL
Stage 2 — no planning commentary shipped	Verified (static)	7 tests; reproduced the bug against a reverted module
Stage 3 — cache, truncation, pruning	Verified (static)	17 tests; token savings measured offline
Stage 3b — narration budget	Partly verified	31 tests; scalar branch observed live, tabular branch not
Stage 4 — window widening	Verified (live)	21 tests; ladder observed firing in production
Stage 5 — report routing	Verified (live)	11/11 through the live router
Stage 5 — report failure handling	Verified (live)	Recovered from a real database outage
Stage 5 — report SQL execution	NOT VERIFIED	No query has run; database unreachable
Token auto-refresh	Verified (static)	13 tests; cannot activate without credentials

6. Recommended next steps

#	ACTION	WHY
1	Fix `DB_PORT` and database credentials	Unblocks the only substantial untested surface — the eleven ported SQL queries
2	Execute each of the eleven reports once against real data	Placeholder arity and tenant binding are covered statically; column names, joins and type casts are not
3	Set the service-account credentials	Activates the token refresh and makes the latency harness usable
4	Re-run the latency harness for a clean baseline	Five stages are now stacked with no end-to-end measurement behind any of them
5	Resolve findings 3, 4 and 5	Two are user-facing correctness; one is a live credential

Appendix — components delivered

NEW	PURPOSE
reports/engine.py	Read-only execution layer: RLS, timeout, bound parameters
reports/definitions.py	Eleven ported SQL reports
endpoints/window_widener.py	Widening planner for empty recent windows
auth/service_token.py	Auto-refreshing service-account token
7 test modules	210 tests

MODIFIED	CHANGE
agents/finance_agent.py	Re-vendored; 43 soft-delete filters restored

MODIFIED	CHANGE
sql_fallback/sql_engine.py	Forced answer on both exits; cache; truncation; pruning
sql_fallback/cost_optimizer.py	Zero-cost complexity resolver
reasoning/claude_reasoner.py	Payload-conditional prompt and token budget
orchestrator/gemini_brain_runner.py	Widening; report dispatch; budget at fallback narration
tools/registry.py · schemas.py	Eleven report tools; 51 → 62
config/constants.py · settings.py	Budgets, thresholds, service-account settings

Prepared with Claude Code. All figures in this report were produced by the runs described; nothing is projected or estimated.